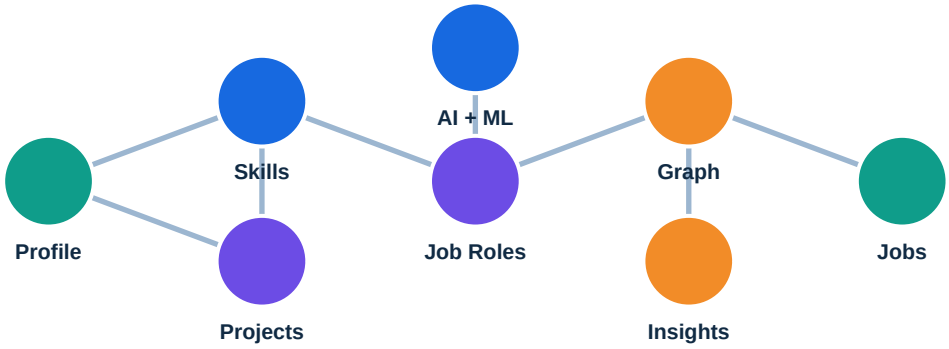
NETGRAPH

Discover Skills. Explore Connections. Find Opportunities.



A Data Mining Based Career & Job Matching Platform

A complete academic project report describing the product vision, requirements, architecture, data model, machine-learning and graph-mining methods, analytics, implementation roadmap, evaluation, privacy and future scope.

Prepared as a project planning and implementation blueprint

Project name: NETGRAPH
Domain: Data Mining • Machine Learning • Graph Analytics • Career Technology

Executive Summary

NETGRAPH is a data-driven career exploration and job-matching platform that analyses a user's education, skills, experience, projects, certifications, resume and optional professional profile links. It compares verified profile information with a curated job-posting dataset to surface potential job matches, explain skill alignment, identify skill gaps, and show related career paths. Its distinguishing capability is a graph-based view of relationships between skills, technologies, projects and job roles, supported by data-mining methods such as clustering, classification, association analysis, anomaly detection and graph metrics. The platform can optionally generate a professional resume from information supplied and confirmed by the user. NETGRAPH provides evidence-based insights from available data; it does not guarantee interviews, employment or job offers.

1. Project Overview

1.1 Problem Statement

Students, graduates and early-career professionals often have information spread across resumes, coding profiles, project repositories, certificates and professional networking sites. They may not know how their current skills map to job requirements, which skills are missing for a target role, or how related roles connect. Conventional job boards primarily list vacancies and may not provide transparent skill-gap analysis or explainable career insights. NETGRAPH aims to consolidate user-approved information, normalize it into a consistent profile, and analyse it against a traceable job dataset using data mining and graph analysis.

1.2 Aim

To develop an accessible platform that converts user-provided career information and job-market data into explainable job-match insights, skill-gap analysis, career-path exploration, graph visualizations and optional resume generation.

1.3 Objectives

- Build a structured profile for personal, educational, professional and project information.
- Allow optional resume upload and optional links to LinkedIn, GitHub, GeeksforGeeks, LeetCode, TopRanker, portfolios and other relevant sites.
- Extract and normalize skills and project evidence, with user review before saving.
- Match profiles to job postings and show contributing factors, matching skills and skill gaps.
- Apply data-mining techniques to discover skill clusters, co-occurrence patterns, job-role groups and anomalies.
- Create an interactive graph connecting skills, technologies, projects and roles.
- Offer personalized career guidance, learning-roadmap suggestions, saved jobs, application tracking and downloadable reports.
- Provide an optional resume builder with templates, preview, editing and PDF/DOCX export.

1.4 Intended Users

User group	Primary needs
Students	Internships, entry-level roles, project portfolio, skill development.
Graduates / job seekers	Role discovery, profile-to-job alignment, skill gaps and application tracking.
Early-career professionals	Adjacent role exploration, upskilling and profile improvement.
Career mentors / administrators	Aggregated dataset insights, trends and guidance, subject to access controls.

2. Scope and Product Principles

- **Profile-first workflow:** users can enter information manually, upload a resume, connect permitted data sources, or combine these methods.
- **Optional means optional:** a user can analyse a profile without a resume; the Resume Builder is available from a clearly marked optional sidebar item.
- **Explainable results:** show the skills, experience and evidence that contributed to a match rather than only a single score.
- **Evidence-aware:** distinguish user-entered facts, extracted information, inferred relationships and job-market observations.
- **Consent and control:** users choose which links to analyse, review extracted data, and can edit or delete their information.
- **Dataset transparency:** display the source and last-updated date for job data; do not imply that historical postings are active vacancies.

Out of scope for the initial version: guaranteeing job offers, automated hiring decisions, scraping restricted websites, making unsupported claims about a person's employability, and providing unverified live vacancies.

3. Functional Requirements

3.1 Account and Profile

- Account registration, sign-in, password reset, profile settings and secure session management.
- Personal details: name and email required; phone, photo, location and other contact details optional and purpose-limited.
- Education: institution, qualification, field of study, dates, grade/CGPA optional, and multiple education entries.
- Experience: role, organization, employment type, dates, responsibilities, achievements and internship details; multiple entries supported.
- Skills: technical and soft skills, user-selected proficiency (optional), evidence source and last verified date.
- Projects: title, description, role, technologies, repository/demo link, dates and outcomes.
- Certifications: title, issuer, credential link/ID if user chooses, and completion date.
- Preferences: desired job roles, domain, location, remote/hybrid/onsite, employment type, salary range and availability; sensitive or unnecessary fields should be omitted.

3.2 Resume Upload and Resume Builder

Resume upload is an optional input in profile creation. Supported formats can include PDF and DOCX, with size and file-type validation. NETGRAPH can parse the document to suggest structured fields, but must show extracted content for user correction. If the user has no resume, they can skip upload and optionally open **Resume Builder** from the left sidebar. The builder collects or reuses confirmed profile data, offers accessible templates, allows section ordering and manual editing, provides a live preview, and exports PDF and DOCX. The user controls whether to include a photo, links, projects, certifications, summary and other sections. A photo is optional and should never be required to create a resume.

Resume Builder capability	Description
Entry point	Optional left-sidebar item labelled "Resume Builder"; also a contextual prompt after profile completion.
Templates	Modern, professional, minimal and creative layouts; readable typography and consistent spacing.
Content	Summary, education, experience, skills, projects, certifications, links and optional achievements.
Customization	Edit text, reorder sections, choose template and include/exclude optional sections.
Preview and export	Preview before export; download PDF and DOCX; retain an editable version if user opts in.

Resume Builder capability	Description
Quality checks	Flag missing dates, inconsistent formatting, overly long sections and unverified claims; never invent credentials.

3.3 Optional Profile Links and Integrations

Provide distinct optional URL fields for LinkedIn, GitHub, GeeksforGeeks, LeetCode, TopRanker, personal portfolio, Kaggle and “Other links” (repeatable). A URL alone is a reference, not proof that the platform can be read. Use official APIs, OAuth or user-provided exports where permitted. For restricted or unavailable sources, ask the user to paste relevant information or upload an export. Show the source, fields retrieved and analysis timestamp; allow the user to review and remove imported details.

Source	Potential analysis, only when accessible and authorized
LinkedIn	User-entered professional headline, experience, education, skills and public/user-provided profile details. Do not assume unrestricted profile scraping.
GitHub	Public or authorized repositories, README descriptions, languages, topics, contributions and project links; avoid treating repository language as confirmed proficiency.
GeeksforGeeks	User-provided profile details or supported/authorized achievement data, such as practice or contest summaries.
LeetCode	User-provided or officially accessible problem-solving/contest summaries; use as supporting evidence, not a complete measure of ability.
TopRanker	User-provided or authorized profile/achievement information where available.
Portfolio / Other links	Project descriptions, demos, articles, certificates or personal websites, subject to safe retrieval and consent.

3.4 Profile Analysis and Results

- Validate inputs, parse optional resume, extract skills/entities and normalize spelling, aliases and categories.
- Display extracted profile summary with edit/confirm controls and a source label for each item.
- Generate a profile completeness indicator without blocking analysis when optional fields are missing.
- Compute potential job matches from skill overlap, related skills, experience alignment, education requirements and user preferences.
- Display matching skills, missing or under-evidenced skills, job source/date, location, work mode and application link when available.
- Provide job filters, sorting, saved jobs and application status tracking.
- Present related roles, career transition paths, learning roadmap and graph-based explanations.
- Generate downloadable analysis reports and allow users to compare their own profile snapshots over time.

4. User Experience and Frontend Design

4.1 Main Navigation

Sidebar section	Purpose
Dashboard	Overview of profile, recent analysis, job matches and trends.
My Profile	Create, edit and verify personal, education, experience, skills and preference data.
Job Matches	Potential matches, filters, match explanations, saved jobs and application tracker.
Skill Insights	Skill coverage, missing skills, demand patterns and development priorities.
Graph Explorer	Interactive network of skills, technologies, projects and job roles.
Career Paths	Related roles, transition routes and evidence-backed role requirements.
Learning Roadmap	Ordered learning goals, resources and progress tracking.
Project Portfolio	Project evidence and technologies; manual entry and authorized repository import.
Resume Builder (Optional)	Create, preview, edit and export a resume if the user wants one.
Saved Jobs / Application Tracker	Manage opportunities and record application stages.
Reports	Download profile, skill and matching analysis.
Settings	Consent, connected links, privacy, export/delete data and preferences.

4.2 Profile Creation Flow

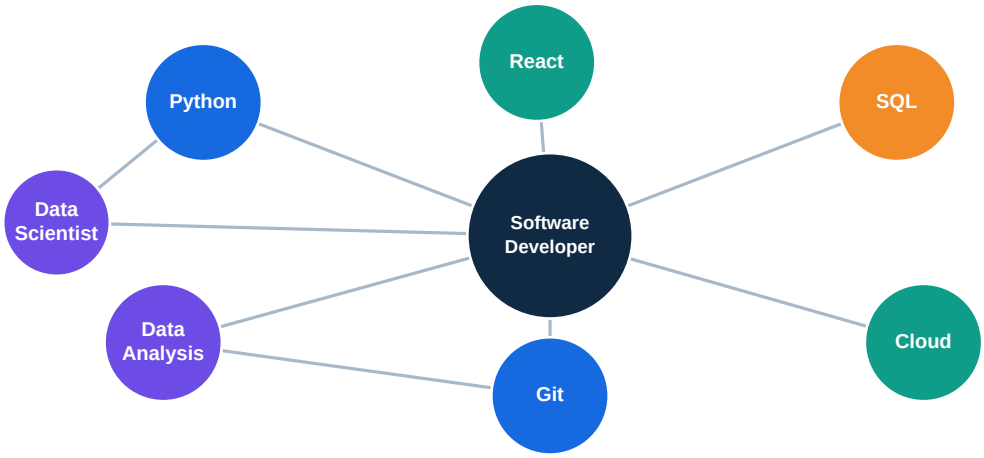
Step	Screen / action	Outcome
1	Basic information	Name, email, location and optional photo/contact details.
2	Education and experience	One or more education and work entries.
3	Skills and projects	Skills, proficiency if known, projects and certifications.
4	Links and resume (optional)	Paste profile URLs and/or upload a resume; skip if not needed.
5	Preferences	Roles, domains, location, work mode, employment type and optional salary range.
6	Review and analyse	Verify extracted/imported details, select data sources and run analysis.

4.3 Analytics and Graph Views

The analytics area should combine concise summary cards with charts that support investigation. Use actual computed results from the selected dataset and show sample/demo data only when clearly labelled.

- **Overview cards:** number of potential matches, profile completeness, verified skills, skill gaps and related roles.
- **Skill comparison chart:** user skills versus frequency/demand in the selected job dataset, with dataset scope and time period.
- **Skill-gap table:** skills requested by target roles, evidence found in profile, and suggested next steps.
- **Role distribution:** job posting counts by role, seniority, location, work mode or industry where data exists.
- **Trend analysis:** changes in skill mention frequency across dated postings; avoid interpreting raw mentions as guaranteed future demand.
- **Match explanation:** per-job factor contributions, matched/missing skills and evidence links.
- **Graph Explorer:** zoom, pan, search, filter by node/edge type, expand neighbours, select a node and inspect supporting postings.

- **Graph analytics:** degree/centrality, communities, shortest paths and skill co-occurrence, with clear definitions.



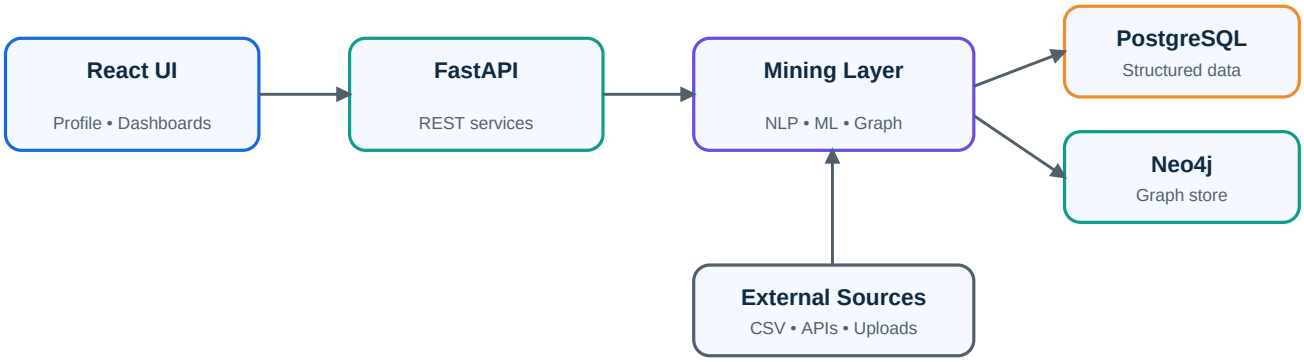
Illustrative relationship graph: a job role is connected to technologies and skills. In the implemented system, nodes and edges must be generated from verified profile evidence and the job dataset, not manually fabricated.

4.4 Accessibility and Responsive UI

- Responsive layouts for desktop, tablet and mobile; keyboard-accessible forms and graph alternatives.
- Consistent labels, clear validation errors, sufficient contrast, readable charts and non-colour-only indicators.
- Empty, loading, success and error states for every data-intensive page.
- Graph selection should have an equivalent text/table view for accessibility and small screens.
- Keep the interface professional, uncluttered and consistent; avoid relying on decorative animations.

5. System Architecture

A modular architecture separates the web interface, API, data processing, mining services and persistence layers. For a student project, a modular monolith is easier to build and test than many independent microservices; modules can be separated later if scale requires it.



Layer	Suggested technology	Responsibilities
Frontend	React + TypeScript, Tailwind CSS, React Router	Forms, profile review, dashboards, charts, graph exploration and reports.
API	Python + FastAPI, Pydantic	Authentication, validation, orchestration, access control and REST endpoints.
Processing	Pandas, NumPy, spaCy / regex	File parsing, cleaning, entity/skill extraction, normalization and feature preparation.

Layer	Suggested technology	Responsibilities
Mining	Scikit-learn, NetworkX, Shogun (where compatible)	Classification, clustering, semi-supervised experiments, association analysis and graph metrics.
Structured store	PostgreSQL	Users, profile facts, job postings, preferences, analysis runs and provenance.
Graph store	Neo4j or NetworkX initially	Skill/role/project relationships and graph queries; Neo4j for persistent graph exploration.
Visualization	Cytoscape.js, Recharts	Interactive network graph and analytical charts.
Testing / delivery	pytest, Vitest/RTL, Postman, Docker, GitHub	Automated tests, API verification, reproducible environment and version control.
Deployment	Vercel + Render / equivalent	Host frontend and API; use managed database services where available.

5.1 Data Flow

- User submits profile fields and chooses whether to upload a resume or connect optional profile links.
- API validates consent, URLs and file type/size; raw files and imported data are handled under retention rules.
- Parsing and extraction produce candidate facts with confidence and source provenance.
- User reviews and confirms extracted fields; confirmed data becomes the analysis profile.
- Job dataset is validated, cleaned, deduplicated, normalized and annotated with source/date.
- Feature engineering builds profile and posting vectors; matching and mining jobs execute.
- Results are stored with model/version metadata, then served to dashboards, graph views and reports.

6. Data Sources and Data Management

6.1 Job Dataset

Start with a permitted, static public dataset of job postings to make experiments repeatable. Fields may include posting ID, job title, description, skills, employer (if permitted), location, work mode, seniority, date posted, source and application URL. For live vacancies, use permitted APIs or licensed feeds and refresh timestamps. Do not present expired or historical records as currently open.

6.2 Data Quality

- Validate required columns, types, encoding and missing values; reject malformed records with a clear report.
- Deduplicate exact and near-duplicate postings using IDs, normalized text and similarity thresholds.
- Normalize skill aliases (for example, common abbreviations and spelling variants) while preserving original text.
- Retain source, collection date, licence/permission notes and transformation history.
- Separate profile-derived skills from skills inferred from a job description; store evidence and confidence.
- Use synthetic/demo records only for interface prototyping and label them as such.

6.3 Example Core Entities

Entity	Example fields
User	id, email, authentication metadata, created_at, settings
Profile	user_id, location, summary, target_roles, work_mode, availability
Education	profile_id, institution, qualification, field, dates, grade_optional

Entity	Example fields
Experience	profile_id, organization, role, dates, description, evidence_source
Skill	id, canonical_name, category, aliases
ProfileSkill	profile_id, skill_id, proficiency_optional, source, confidence, verified
Project	profile_id, title, description, technologies, repo_url, demo_url
Certification	profile_id, title, issuer, date, credential_url_optional
ExternalLink	profile_id, provider, url, consent_status, last_checked
JobPosting	id, title, description, skills, location, work_mode, posted_at, source_url
AnalysisRun	profile_id, dataset_version, model_version, timestamp, status, metrics
MatchResult	analysis_id, job_id, score, factor_breakdown, matched_skills, gaps
GraphNode / GraphEdge	node_type, canonical_id, relationship_type, weight, evidence_count

7. Data Mining and Machine Learning Methods

Select algorithms according to data volume, labels and evaluation feasibility. A baseline should be implemented first; advanced methods should be added only when they improve measurable performance or provide a meaningful research comparison.

Technique	Possible method/tool	Use in NETGRAPH
Text processing	spaCy, regex, Pandas	Extract skills, role titles, qualifications and experience evidence from permitted text.
Feature engineering	TF-IDF, multi-hot skill vectors, embeddings if feasible	Represent profile and job-posting content for similarity and modelling.
Job matching	Weighted skill overlap, cosine similarity, rules	Rank potential posting alignment and expose factor-level explanations.
Clustering	K-Means, hierarchical clustering; Scikit-learn	Discover groups of related skills, job descriptions or role profiles.
Classification	Random Forest, SVM, Logistic Regression	Classify postings into defined job-role categories when labelled data exists.
Semi-supervised learning	Label propagation / self-training, Scikit-learn	Explore partially labelled job-role or skill-category data; validate assumptions.
Multitask learning	Shogun, if compatible; compare shared-task baselines	Experiment with related classification tasks, such as role families or skill categories.
Association mining	Apriori / FP-Growth implementation	Find skill combinations and report support, confidence and lift.
Anomaly detection	Isolation Forest, Local Outlier Factor	Flag unusual records or skill combinations for review, not label users negatively.
Graph mining	NetworkX / Neo4j	Measure degree, centrality, communities, paths and relationships between skills and roles.
Large-scale learning	MLPY / Dlib only after compatibility review; scalable Scikit-learn pipelines	Benchmark only where dataset size and supported interfaces justify it.

7.1 Matching Score Design

A transparent score may combine normalized skill coverage, related-skill proximity, experience alignment, education requirements and stated preferences. Example conceptual form: Match = w■(Skill coverage) + w■(Related skills) + w■(Experience alignment) + w■(Preference fit), where weights are selected through validation and shown in the report.

Do not imply the score is a probability of receiving an offer. Missing or unobservable fields should not silently count as negative evidence.

7.2 Evaluation Metrics

Task	Metrics / checks
Classification	Precision, recall, F1, confusion matrix; compare with majority-class baseline.
Clustering	Silhouette score, Davies–Bouldin index, cluster interpretability and stability.
Association rules	Support, confidence, lift, minimum sample count and rule stability.
Matching	Precision@K / Recall@K where relevance labels exist, ranking review, coverage and explanation quality.
Graph	Node/edge counts, connected components, degree distribution, community quality and query performance.
Data extraction	Manually annotated sample, precision/recall by entity type, confidence calibration and error review.
System	API latency, throughput under test load, upload validation, authorization tests and UI usability feedback.

8. Analytics and Reporting

- **Personal analytics:** profile completeness, verified skills, project/certification counts, analysis history and user-controlled change over time.
- **Job-market analytics:** skill frequency, role distribution, location/work-mode distribution and posting freshness within the dataset.
- **Skill analytics:** skills by category, co-occurrence pairs, gaps for selected roles, and evidence coverage.
- **Graph analytics:** most connected skills, communities, paths between a current skill and a target role, and role neighbourhoods.
- **Recommendation analytics:** reason codes for each suggestion, source postings and suggested learning objectives.
- **Export:** downloadable PDF report containing input summary, dataset scope, match explanations, charts, skill gaps, graph snapshot and methodology.

Reports should clearly show analysis date, dataset version/date range, model version, missing-data caveats and whether a result is inferred or directly observed. Avoid ranking people against one another unless a carefully governed, consented research purpose exists; individual guidance should focus on the user's own goals.

9. Backend API Design

Endpoint (illustrative)	Purpose
POST /api/auth/register, /login	Account and session management.
GET/PUT /api/profile	Read or update current user's profile.
POST /api/profile/resume	Upload optional resume for parsing.
POST /api/profile/links/validate	Validate user-submitted profile URLs and consent selection.
POST /api/profile/links/analyse	Fetch permitted profile data or process user-provided exports.
POST /api/profile/extraction/confirm	Save user-confirmed extracted facts.
POST /api/analysis/run	Start a profile-to-job and data-mining analysis.
GET /api/analysis/{id}	Return run status, metadata and summary.

Endpoint (illustrative)	Purpose
GET /api/matches	Return paginated job matches and filters.
GET /api/matches/{job_id}	Return match factors, evidence, gaps and posting provenance.
GET /api/skills, /api/skills/{id}/related	Search skills and related graph neighbours.
GET /api/graph	Return graph nodes and edges under filters.
GET /api/analytics/overview	Return overview cards and chart series.
GET /api/analytics/skills, /roles, /trends	Return domain-specific analytics.
POST /api/resume/generate	Generate a resume from confirmed user data and selected template.
GET /api/reports/{id}/download	Download an authorized report.
POST /api/jobs/{id}/save, /applications	Save a job or update application status.
DELETE /api/profile/data	Delete profile data and related stored artifacts subject to policy.

All endpoints require authorization checks, input validation, rate limiting where appropriate, pagination for large results and consistent error responses. External link analysis should be asynchronous where latency or provider limits require it.

10. Security, Privacy and Responsible Use

- Request clear, specific consent before reading or processing external profile data; consent can be withdrawn.
- Do not scrape LinkedIn or any platform in ways that violate its terms. Prefer official APIs, OAuth scopes or user-supplied exports.
- Use HTTPS, secure password hashing, access controls, secret management and least-privilege database credentials.
- Validate uploads by extension, MIME type and content; scan where available; enforce size limits and store outside public web paths.
- Prevent SSRF when processing user URLs: restrict schemes, block private/internal network destinations, limit redirects and response sizes, and use timeouts.
- Minimize stored personal data; encrypt sensitive fields where appropriate; define retention, export and deletion flows.
- Keep profile information private by default; never expose a user's resume, links or analysis to other users without explicit authorization.
- Do not infer protected or sensitive traits, and do not use the system as an automated hiring decision-maker.
- Show uncertainty and provenance; let users correct extraction mistakes and distinguish observed data from model inference.
- Use accessible alternatives to charts and graphs and provide understandable explanations for recommendations.

11. Non-Functional Requirements

Area	Target / requirement
Usability	A new user can create a basic profile and run analysis without filling optional fields.
Performance	Paginate results; cache appropriate aggregates; report measured latency after testing rather than promising an unverified target.
Reliability	Graceful failures for unavailable integrations, malformed files, model errors and empty datasets.

Area	Target / requirement
Scalability	Modular processing pipeline, background jobs for large analysis, indexed database fields and graph query limits.
Maintainability	Typed API schemas, modular services, documented environment configuration and code review.
Reproducibility	Dataset snapshot/version, preprocessing configuration, random seeds and model version stored per run.
Accessibility	Keyboard navigation, contrast, labels, text alternatives and responsive layout.
Security	Authentication, authorization, input validation, safe upload and URL processing, audit logging.
Portability	Docker-based local setup and environment-specific deployment configuration.

12. Implementation Plan

Phase	Work	Deliverable
1. Planning	Confirm requirements, user flow, dataset licence, research questions and scope.	Requirements specification and approved dataset.
2. Foundation	Set up Git repository, React app, FastAPI service, PostgreSQL and development environment.	Running skeleton and database migrations.
3. Profile UX	Build authentication, profile form, repeatable education/experience/projects, preferences and optional links.	Profile creation and editing.
4. Resume handling	Implement optional PDF/DOCX upload, parser, extraction review and Resume Builder with export.	Resume parsing and optional resume generation.
5. Dataset pipeline	Validate, clean, deduplicate and normalize job records; preserve provenance.	Reproducible cleaned dataset.
6. Extraction and features	Skill dictionary/ontology, NLP extraction, aliases, feature vectors and review workflow.	Profile/job skill extraction.
7. Baseline matching	Weighted matching and explainable result details; add evaluation labels if possible.	Initial job match results.
8. Mining and graph	Clustering, association analysis, classification experiment, graph construction and graph metrics.	Analytics and Graph Explorer.
9. API and dashboard	Connect endpoints, charts, filters, saved jobs, tracker and reports.	Integrated frontend and backend.
10. Testing and deployment	Unit/integration/security/usability tests, Docker, deployment, documentation and demo.	Release candidate and academic report.

12.1 Suggested 10-Week Schedule

Week	Focus
1	Scope, literature review, data source selection and UI wireframes.
2	Project setup, database schema, authentication and profile forms.
3	Resume upload/parser, optional links, consent and extraction review.
4	Job dataset cleaning, normalization and NLP skill extraction.
5	Baseline matching, explainable scores and first evaluation.
6	Clustering, classification or semi-supervised experiment; association mining.
7	Graph construction, Neo4j/NetworkX queries and Graph Explorer.
8	Analytics dashboard, career paths, learning roadmap and resume builder.
9	Integration, testing, security checks, usability feedback and fixes.
10	Deployment, final report, presentation, demo script and results discussion.

13. Testing Strategy

- **Unit tests:** parsers, validators, normalization, matching factors, graph builders and scoring functions.
- **API tests:** authentication, authorization, request validation, pagination, file upload and deletion workflows.
- **Frontend tests:** form validation, optional field behaviour, profile review, graph filters, charts and resume builder controls.
- **Integration tests:** profile → extraction → confirmation → analysis → results → report export.

- **Security tests:** access-control boundaries, unsafe URLs, upload validation, injection handling, secret leakage and rate limits.
- **ML tests:** baseline comparisons, train/test separation, label quality, class imbalance, data leakage checks and error analysis.
- **Usability tests:** ask representative students to complete profile setup, interpret match explanations and navigate graph views.
- **Regression tests:** retain fixed sample cases and verify results after changes to the pipeline or model.

14. Expected Results and Deliverables

- Working responsive NETGRAPH web application with profile creation, optional resume upload and optional external links.
- Optional Resume Builder with templates, editable preview and PDF/DOCX export from user-confirmed information.
- Dataset ingestion and reproducible preprocessing pipeline with data provenance.
- Skill extraction and normalization module with user review.
- Job matching results with transparent explanations, matched skills and skill gaps.
- Analytics dashboard with skill, role, trend and profile insights.
- Interactive graph visualization for skill, technology, project and job-role relationships.
- Career-path recommendations and learning roadmap grounded in the dataset.
- Saved jobs, application tracking and downloadable analysis reports.
- Test suite, model evaluation results, API documentation, deployment guide, source repository, final report and presentation.

15. Limitations and Risk Management

Risk / limitation	Mitigation
Restricted profile platforms	Use authorized APIs or user-supplied data; clearly indicate unavailable fields.
Noisy skill extraction	Use a curated skill vocabulary, confidence labels, user correction and sampled evaluation.
Biased or incomplete job dataset	Document source and coverage; avoid broad labour-market claims beyond the dataset.
Outdated job postings	Display posted/checked dates and use permitted refreshed sources for live vacancies.
Misleading match scores	Explain factors, validate ranking where possible and state that scores are not hiring probabilities.
Resume parsing errors	Preview extracted text, highlight uncertain fields and require user confirmation.
Algorithm/library compatibility	Build a baseline with maintained tools; test Shogun/MLPY/Dlib independently before committing.
Privacy or URL risks	Consent, minimization, safe URL fetching, strict authorization and deletion controls.
Scope growth	Prioritize a complete baseline workflow; defer integrations and advanced models until core is stable.

16. Future Enhancements

- Authorized additional integrations with professional portfolios, learning platforms and job APIs.
- Multilingual resume parsing and skill taxonomies.
- Semantic embeddings for richer similarity, compared against interpretable baselines.

- Personalized learning-resource integration and progress evidence.
- Time-series analysis of job-skill mentions with sufficient dated data.
- More advanced graph embeddings or graph neural network experiments if dataset scale and labels justify them.
- Mentor-facing views with user-controlled sharing and privacy-preserving aggregation.
- Mobile-first companion experience and notifications for new matching postings.

17. Conclusion

NETGRAPH combines a practical career-support workflow with a substantive data-mining foundation. Users can enter profile details, optionally upload an existing resume, or optionally provide links to professional and coding profiles. They can also skip resume upload and use the optional Resume Builder to create one from confirmed information. The platform then analyses profile evidence against a documented job dataset, explains potential matches, identifies skill gaps, surfaces related roles and presents relationships through interactive graphs and analytics. Its value depends on transparent data provenance, user control, responsible handling of external platforms, and honest communication of model limitations. A phased implementation beginning with a reliable profile, dataset pipeline, explainable baseline matcher and graph view provides a feasible academic project, while advanced mining methods and integrations can be added and evaluated incrementally.

Appendix A. Recommended Frontend Pages

Page	Key components
Dashboard	Overview cards, recent analysis, match summary, skill chart, quick actions.
My Profile	Personal info, education, experience, skills, projects, certifications, preferences, links and optional resume upload.
Profile Review	Extracted facts, source/confidence labels, edit/confirm controls and consent summary.
Job Matches	Search/filter/sort, score explanation, job details, skill overlap/gaps, save and track.
Skill Insights	Demand comparison, skill-gap priorities, related skills and learning actions.
Graph Explorer	Search, node/edge filters, neighbour expansion, node detail and accessible table view.
Career Paths	Target roles, related roles, transition skills and roadmap.
Project Portfolio	Project cards, technologies, authorized GitHub evidence and manual edits.
Resume Builder	Optional sidebar entry, template selector, section editor, preview and export.
Reports	Analysis history, dataset/model metadata, chart snapshots and PDF export.
Settings	Profile visibility, consent, connected links, data export/deletion and account controls.

Appendix B. Recommended Initial Data Mining Experiments

Experiment	Research question	Comparison / output
Skill co-occurrence	Which skills are repeatedly requested together?	Support, confidence, lift; compare thresholds and inspect examples.
Role clustering	Do job postings form interpretable role groups?	K-Means vs hierarchical clustering; silhouette and human review.
Role classification	Can posting text predict predefined role families?	Baseline vs Random Forest/SVM; F1 and confusion matrix.

Experiment	Research question	Comparison / output
Profile-job matching	Does a transparent score retrieve relevant postings?	Weighted overlap vs cosine similarity; Precision@K if relevance labels are available.
Graph structure	Which skills bridge role communities?	Degree/betweenness/community summaries with evidence counts.
Semi-supervised labels	Can limited labels support role/skill categorization?	Label propagation vs supervised baseline, with held-out evaluation.

Appendix C. Example Result Explanation

Illustrative only: “This posting aligns with several skills confirmed in your profile, including Python, SQL and React. The description also requests cloud deployment and system design, which were not found in the information you supplied. The match score summarizes selected profile-to-posting factors and is not a probability of interview selection or employment. Review the source and posting date before applying.”

Appendix D. Implementation Notes

- Use a single canonical skill vocabulary with aliases and categories, but retain raw extracted terms for audit and correction.
- Store model version, dataset snapshot, feature configuration and timestamp for every analysis run.
- Use background processing for slow parsing/mining tasks and expose status to the frontend.
- Keep external-source adapters independent so unsupported platforms can be disabled without breaking profile creation.
- Provide a no-integration fallback: manual entry and user-provided resume/profile exports must remain fully supported.
- Begin with a compact dataset and a complete end-to-end flow before adding large-scale or deep-learning components.

End of report • NETGRAPH